Assignment 4 solution

1. 【考点：CPU的功能和基本结构】在经典的冯·诺依曼架构 CPU 中，关于程序计数器（PC）和指令寄存器（IR）在指令周期中的状态与作用，下列说法绝对正确的是（ ）

   A. 取指周期完成时，PC 中保存的始终是当前正在执行指令的绝对物理首地址
   B. 无论是执行算术运算指令还是转移指令，指令的操作码字段均在 IR 中完成译码，并且 IR 的内容会保持至该条指令执行结束
   C. 在间接寻址的执行周期中，CPU 需要将 PC 的值再次送往主存地址寄存器（MAR）以获取有效地址
   D. 若当前指令发生未定义指令异常（Undefined Instruction），硬件系统将直接把 IR 中的值压入内核栈中以保存现场

   【答案】：B
   【解析】：
   A选项错误：取指周期结束（或取指过程中），PC 的值会自动更新为下一条将要执行指令的地址，而不是当前指令的地址
   C选项错误：间接寻址获取有效地址时，送往 MAR 的是 IR 中的形式地址字段或通用寄存器中的值，而不是 PC 的值。PC 仅用于指示指令地址
   D选项错误：发生异常或中断时，硬件保护现场压入栈中的断点地址是 PC 的值，而不是 IR 的值
   B选项正确：IR 用于存放当前正在执行的指令，操作码译码由指令译码器（ID）完成，且为了保证执行阶段控制信号的稳定，IR 的内容必须保持到指令执行周期结束

2. 【考点：数据通路的功能和基本结构】某 CPU 内部采用单总线数据通路结构，该总线连接了所有通用寄存器、ALU 的输入/输出暂存器以及主存数据寄存器（MDR）。在执行一条如 `ADD R1, R2`（将 R1 与 R2 相加，结果存入 R1）的指令时，在不考虑取指和译码阶段的前提下，仅其执行阶段至少需要几个 CPU 时钟周期？

   A. 1 个
   B. 2 个
   C. 3 个
   D. 4 个

   【答案】：C
   【解析】：
   单总线结构意味着在同一个时钟周期内，总线上只能传输一个数据。`ADD R1, R2` 的执行阶段需要三步冲突的传输操作：
   时钟周期 1：将 R1 的值通过总线送入 ALU 的输入暂存器 A（因单总线无法同时送两个源操作数，必须暂存一个）
   时钟周期 2：将 R2 的值通过总线送往 ALU 的另一个输入端 B，ALU 进行加法运算，结果存入 ALU 输出暂存器 Z
   时钟周期 3：将输出暂存器 Z 中的运算结果通过总线写回寄存器 R1
   因此至少需要 3 个时钟周期

3. 【考点：控制器的功能和工作原理】在微程序控制器设计中，关于微指令、微程序和机器指令之间的关系，下列描述错误的是（ ）

   A. 通常情况下，计算机指令系统中一条机器指令的功能是由一段存放在控制存储器（CS）中的微程序来实现的
   B. 机器指令的操作码字段经过微地址形成部件译码后，产生的是对应微程序在控制存储器中的起始微地址
   C. 微程序控制器通过微指令的下地址（Next Address）字段来实现微程序的顺序执行和分支跳转

D. 相比于硬布线控制器，微程序控制器的执行速度更快，因为其控制逻辑完全由高速的 SRAM 存储器阵列实现

【答案】：D
【解析】：
D 选项错误。微程序控制器的核心思想是用“软件（微代码）”的思想来设计硬件控制逻辑，每生成一个控制信号都需要访问一次内部的控制存储器（ROM 或只读 SRAM），因此其执行速度慢于纯硬件组合逻辑电路实现的硬布线控制器。它的优势在于结构规整、易于修改和扩展指令集，而非速度快

4. 【考点：异常和中断机制的响应】当 CPU 在指令执行周期的末尾检测到外部设备发送的中断请求（INTR）并决定响应时，系统会进入硬件层面的“中断隐指令”阶段。下列操作中，不属于中断隐指令自动完成的工作是（ ）

   A. 关中断（将状态寄存器中的中断允许位清零，防止被同级中断打断）
   B. 保存断点（将当前的 PC 和 PSW 压入堆栈）
   C. 保存通用寄存器现场（将程序运行依赖的 R0-R31 等通用寄存器值压入堆栈）
   D. 形成中断服务程序的入口地址（如通过硬件向量法读取中断向量表送入 PC）

   【答案】：C
   【解析】：
   中断隐指令是由硬件自动完成的不可被打断的底层操作，其核心任务只有三个：关中断、保存断点（PC/PSW）、引出中断服务程序（找到入口地址）。而保存通用寄存器现场以及后续的恢复现场，是由操作系统通过软件指令在中断服务程序（ISR）的首尾部分编写代码完成的

5. 【考点：异常和中断的分类】在现代处理器架构中，异常（Exception）和中断（Interrupt）被严格分类。下列事件中，与其他三项在触发源头和系统处理逻辑上存在本质区别的是（ ）

   A. 应用程序执行系统调用指令（如 `syscall` 或 `int 0x80`）请求陷入内核态
   B. 虚拟存储管理系统中，CPU 访问的逻辑页面对应的页表项有效位为 0 触发缺页异常
   C. CPU 算术逻辑单元在执行带符号数乘法指令时发生严重的数据溢出
   D. 网络接口卡（NIC）在成功接收到一个数据包后向 CPU 发送信号

   【答案】：D
   【解析】：
   A、B、C 均为内部异常（Exception/内部中断）。它们的特点是同步的，由 CPU 内部当前正在执行的指令本身引起（A 为陷入 Trap，B 为故障 Fault，C 为终止或故障）
   D 属于外部中断（Interrupt/外部硬件中断）。其特点是异步的，由 CPU 外部的外设（如网卡、磁盘等）触发，与当前 CPU 正在执行的指令毫无逻辑关系

6. 【考点：指令流水线】在经典的五级（IF, ID, EX, MEM, WB）指令流水线中，硬件已经实现了完全数据旁路（Full Forwarding / Bypassing）技术。对于下列四组存在数据相关（RAW 冒险）的汇编指令序列，即使启用了旁路技术，必然仍会引起流水线阻塞（Stall） 的指令组合是（ ）

   A. `ADD R1, R2, R3` 紧接着 `SUB R4, R1, R5`（均是寄存器算术运算）
   B. `LOAD R1, 0(R2)` 紧接着 `ADD R3, R1, R4`（先访存读，再运算）
   C. `ADD R1, R2, R3` 紧接着 `STORE R1, 0(R4)`（先运算，再访存写）
   D. `LOAD R1, 0(R2)` 紧接着 `STORE R1, 0(R3)`（先访存读，再访存写）

   【答案】：B
   【解析】：
   A：ADD 的结果在 EX 段末尾产生，SUB 在 EX 段开头需要，可通过 EX-EX 旁路直接传递，无阻塞
   B：LOAD 指令要到 MEM 段末尾才能从主存取回数据，而紧接的 ADD 指令在 EX 段开头就需要该数据。由于时间上 MEM 段晚于同周期的 EX 段，数据发生了“时光倒流”，因此旁路技术无法完全消除冒险，硬件必须插入 1 个阻塞周期（Stall）
   C：ADD 结果在 EX 末尾产生，STORE 在 MEM 段才需要写入数据，EX-MEM 旁路绰绰有余
   D：LOAD 在 MEM 末尾读取数据，STORE 在 MEM 段写入数据，可通过 MEM-MEM 旁路直接传递，无阻塞

7. 【考点：超标量和动态流水线的基本概念】现代高性能处理器为了榨取指令级并行度（ILP），采用了多种高级流水线技术。下列关于这些技术的描述，错误的是（ ）

   A. 超标量（Superscalar）技术通过在 CPU 内集成多套独立的功能执行部件，使得每个时钟周期可以动态发射（Issue）并执行多条指令
   B. 超长指令字（VLIW）技术高度依赖编译器的静态调度，由编译器在编译阶段将多条无数据冲突的指令打包为一条超长指令，显著降低了硬件动态调度的复杂度
   C. 乱序执行（Out-of-Order Execution）流水线允许指令在资源就绪时提前进入执行阶段，但为了保证程序语义正确和精确异常，必须严格按程序原有顺序提交（In-order Commit）
   D. 超流水线（Superpipelining）技术的核心思想是在不增加时钟频率的前提下，通过在一个时钟周期内多发射几条指令来提升系统的吞吐率

   【答案】：D
   【解析】：
   D 选项描述错误。超流水线（Superpipelining）的核心思想是通过增加流水线的级数（将一级细分为多级逻辑），从而显著提高 CPU 的时钟频率。它在一个时钟周期的单一节拍内依然只发射一条指令，并非“单周期多发射”。“单周期内发射多条指令”是超标量（Superscalar）技术的特征

8. 【考点：指令流水线性能计算】某计算机采用六级指令流水线，各流水段的硬件延迟时间分别为 100 ns, 150 ns, 120 ns, 150 ns, 100 ns, 80 ns，流水线段间锁存器的延迟忽略不计。在不考虑任何数据冒险、控制冒险及分支开销的理想状态下，该流水线连续执行 1000 条指令，其完成任务所需的总时间为（ ）

   A. 150,750 ns
   B. 150,000 ns
   C. 700,750 ns
   D. 100,500 ns

   【答案】：A
   【解析】：
   流水线的时钟周期 T 取决于最慢流水段的时间，即 T = max(100, 150, 120, 150, 100, 80) = 150 ns
   采用 k 级流水线（本题 k = 6）连续执行 n 条指令（本题 n = 1000）的理论总时间公式为：(k + n - 1) × T
   计算代入：(6 + 1000 - 1) × 150 ns = 1005 × 150 ns = 150,750 ns

9. 【考点：多处理器基本概念】在 Flynn 分类法体系中，现代 GPU（图形处理器）和专属 AI 基础设施加速器在执行大规模深度学习张量运算时，其底层硬件并行架构主要基于下列哪种模型演进而满负荷运转?

   A. SISD（单指令流单数据流）：依靠单核极限频率和极深流水线实现串行暴力加速
   B. SIMD（单指令流多数据流）：通过统一的控制部件广播单一指令，驱动成百上千个 ALU 阵列同时对不同的数据元素执行相同的运算（数据级并行）
   C. MISD（多指令流单数据流）：多个处理核对同一个输入数据流执行不同的加密或容错算法
   D. MIMD（多指令流多数据流）：每个物理核心独立取指，依靠复杂的互斥锁和共享内存来协调完全不同的任务控制流

   【答案】：B
   【解析】：
   现代 GPU 架构（包括 CUDA 环境下的 SIMT 单指令多线程机制）和 AI 矩阵计算硬件的核心加速基础正是 SIMD（单指令流多数据流）/ 向量处理器机制。深度学习推理或训练中的矩阵乘加操作，本质上是对海量独立的数据元素执行相同的算术运算指令。SIMD 架构可以用极低的控制电路开销换取极高的数据级并行度（DLP），是当前 AI 计算的最优解

10. 【考点：硬件多线程与多核处理器的基本概念】关于硬件多线程技术和共享内存多处理器（SMP），下列说法正确的是（ ）

A. 细粒度多线程（FGMT）技术只有在当前执行线程遭遇长延迟事件（如 L2 Cache 缺失需要访问主存）时，才会切换到另一个线程执行
B. 粗粒度多线程（CGMT）技术在每一个时钟周期都会轮换发射不同线程的指令，能够有效隐藏流水线中所有的短延迟（如数据冒险）
C. 在对称多处理器（SMP）系统中，所有的 CPU 核心处于平等地位，它们共享同一块物理内存空间，且任意一个核心访问主存中任意地址的理论延迟是相等的（UMA 架构）
D. 同步多线程（SMT / 超线程）技术必须依赖多个物理 CPU 核心才能实现，无法在单一的超标量处理器核心内部位完成多线程并发发射

【答案】：C
【解析】：
A选项错误：遇到长延迟事件才切换的是粗粒度多线程（CGMT）
B选项错误：每个时钟周期都轮换切换的是细粒度多线程（FGMT）
D选项错误：同步多线程（SMT）是在同一个超标量物理核心内部，通过复用发射队列和执行部件，在同一个时钟周期内同时发射不同线程的指令，并不需要多个物理核心
C选项正确：对称多处理器（SMP）采用的是均匀访存模型（UMA），所有处理器对共享主存的访问权利对等且延迟一致

11. 【考点：结构冒险、数据冒险和控制冒险的处理】某高并发 LLM 推理引擎的核心张量计算模块包含如下指令序列，并在标准的 5 段（取指 IF、译码/读寄存器 ID、执行/计算地址 EX、访存 MEM、写回 WB）RISC 流水线处理器上运行。该处理器支持全路数据旁路（Forwarding/Bypassing）技术

```
I1: LOAD  R1, 0(R2)      // 将内存数据加载到 R1
I2: ADD   R3, R1, R4     // R1 和 R4 相加，结果存入 R3
I3: MUL   R5, R3, R6     // R3 和 R6 相乘，结果存入 R5
I4: STORE R5, 0(R7)      // 将 R5 存入内存
```

关于该指令序列在流水线中的数据冒险（Data Hazard）及处理情况，下列说法完全正确的是（ ）

A. 虽然系统采用了全路数据旁路技术，但 I1 和 I2 之间存在“载入-使用（Load-Use）”冒险，硬件仍必须在 I1 和 I2 之间强行插入 1 个时钟周期的阻塞（Stall）或气泡
B. I2 和 I3 之间存在写后读（RAW）冒险，由于乘法运算的执行周期较长，旁路技术无法将 I2 在 EX 段计算出的结果及时送给 I3，因此必然发生流水线停顿
C. I3 和 I4 之间存在数据依赖，I4 必须等待 I3 在 WB 段将结果真正写入 R5 寄存器后，才能在 ID 段读取 R5，这期间需要插入 2 个时钟周期的阻塞
D. 若编译器开启了指令重排优化（Instruction Scheduling），只要在 I1 和 I2 之间插入任意一条与 R1、R2、R4 无关的指令，就能在不使用旁路技术的前提下，完美消除原有的等待周期

【答案】：A
【解析】：
A 选项完全正确（无歧义）：对于 `LOAD R1, 0(R2)` 和随后的 `ADD R3, R1, R4`，ADD 指令在 EX 段（执行阶段）就需要用到 R1 的值；而 LOAD 指令必须在 MEM 段（访存阶段）结束后才能从内存中拿到 R1 的实际数据。此时旁路技术只能将 MEM 段的结果转发给 EX 段，但时间线上已经晚了半个拍子（即“载入-使用”冒险）。因此，硬件必须强行在 ID 段阻塞 ADD 指令 1 个周期，等待 LOAD 进入 WB 段（或通过 MEM-EX 旁路）才能继续。这是经典 5 段流水线无法仅靠旁路消除的唯一数据冒险
B 选项错误：I2（ADD）在 EX 段就能计算出 R3 的结果，而 I3（MUL）也在 EX 段才需要 R3。通过 EX-EX 内部旁路，ADD 的结果可以直接送给 MUL，不需要停顿
C 选项错误：I3（MUL）在 EX 段产生结果，I4（STORE）在 EX 段计算目标地址，并在 MEM 段才真正需要把 R5 的数据写出。通过 EX-MEM 旁路，R5 的数据可以直接传递，无需插入任何阻塞
D 选项错误：如果不使用旁路技术，LOAD 指令必须走到 WB 段写回寄存器后，后续指令才能在 ID 段读到正确数据。这中间有 EX 和 MEM 两个周期的物理间隔，至少需要插入 2-3 个 NOP 才能消除冲突，仅仅插入 1 条无关指令是绝对不够的

12. 【考点：多处理器基本概念】在现代计算基础设施中，底层硬件通常采用高度并行的多处理器架构来加速任务。下列描述中必定错误的是（ ）

A. 在现代共享内存多处理器（SMP）架构中，各个物理核心必须各自独占完全隔离且独立的物理主存空间，以防止高并发场景下缓存一致性协议（如 MESI）引发的总线风暴
B. 向量处理器（Vector Processor）属于典型的 SIMD（单指令流多数据流）架构，它只需发射一条指令，就能同时对分布在寄存器组中的多个数据元素执行相同的操作，非常适合稠密矩阵运算
C. 细粒度硬件多线程（Fine-grained Multithreading）处理器会在每个时钟周期轮换发射不同线程的指令，这种设计能够极大地掩盖指令流水线中的各种冒险延迟，但会牺牲单个线程的绝对执行延迟
D. 同步多线程（SMT / 超线程）技术允许在同一个时钟周期内，将多个不同硬件线程的指令并发地发射到多个空闲的功能部件（如 ALU）上执行，它是对超标量技术的进一步扩展

【答案：A】
【解析】：
A 选项必定错误（即正确答案）：共享内存多处理器（SMP）的核心定义就是所有的物理核心共享同一片物理主存地址空间。它们可以通过相同的物理地址访问同一块内存区域。正是因为共享主存且各自拥有 L1/L2 缓存，才需要引入 MESI 等缓存一致性协议来维护数据状态，如果完全独立隔离就不叫 SMP 了
B 选项正确：这是 SIMD 架构的核心特征，一条指令驱动多个数据通路，在 CUDA 架构底层也有这种思想的深刻体现
C 选项正确：细粒度多线程在每个周期都切换不同线程发射指令，因此没有任何一个线程能在流水线中产生紧密的数据冒险，大幅提升了吞吐量；但代价是单个特定线程的指令在流水线中被稀释，导致单线程执行总时间变长
D 选项正确：SMT（超线程）允许在一个周期内将属于不同线程的指令塞进超标量处理器的多个不同的空闲发射槽（Issue Slots）中，极致压榨硬件资源

13. 【考点：异常和中断机制】在构建分布式计算节点时，CPU 必须具备极强且稳健的异常和中断处理能力。关于 CPU 异常和中断机制的硬件检测与响应逻辑，下列说法存在逻辑错误的是（ ）

A. CPU 对外部中断（如网卡数据到达的 INTR 请求）的检测通常硬连线在每个指令执行周期的末尾；这意味着只要当前指令没有执行完毕，无论外部可屏蔽中断请求多么紧急，CPU 都绝对不会提前响应
B. 当发生严重且不可恢复的硬件故障（如内存奇偶校验错误，属于“终止 Abort”类异常）时，CPU 会将当前引发错误的指令 PC 值压入用户态程序的运行栈（User Stack）中，随后陷入内核态执行崩溃转储
C. 软中断指令（如用于系统调用的 `syscall` 或 `trap`）引发的控制流转移属于内部异常中的“陷阱（Trap）”，其设计初衷就是有意识的程序干预，因此其返回地址通常是该软中断指令的下一条指令
D. 若在执行取指操作时引发了缺页异常（Page Fault，属于“故障 Fault”），操作系统内核在将页面从磁盘调入主存并修复页表后，返回地址必定是引发该缺页异常的那条指令本身，以便 CPU 重新尝试获取该指令

【答案】：B
【解析】：
B 选项存在严重逻辑错误（即正确答案）：当发生异常或硬件中断时，CPU 保存上下文状态（如 PC、寄存器）的区域绝对不可以是用户态的执行栈（User Stack）。为了保证操作系统的安全性和防止恶意程序破坏现场，硬件会自动切换栈指针，将现场信息压入内核栈（Kernel Stack），或者保存在 CPU 内部专用的硬件寄存器中（如异常处理返回寄存器）。此外，Abort 类异常往往由于硬件崩溃，连精确的 PC 值都可能无法提供
A 选项正确：外部中断（INTR）是异步的，CPU 设计为在每条指令的最后一个机器周期（执行周期）结束后去采样中断请求线，不会在指令执行到一半时打断数据通路
C 选项正确：Trap（陷阱）是有意为之的，例如系统调用 `syscall`，内核处理完网络 IO 请求后，理应返回到调用者的下一条语句继续执行

D 选项正确：缺页异常（Fault）的核心机制是“缺啥补啥，补完重做”。因为引发缺页的 LOAD/STORE 指令根本没拿到数据，所以必须返回到该指令本身重新执行一遍访存

14. 【考点：数据通路的功能和基本结构】某系统底层研发团队在设计一款极简的定长指令集 RISC 处理器，其单总线数据通路中所有的寄存器均由统一的全局时钟进行边沿触发同步。关于该处理器的控制器与数据通路工作原理，下列描述完全正确的是（ ）

A. 在微程序（Microprogrammed）控制器中，微指令存放在控制存储器（CM）中。为了突破访问延迟瓶颈，现代高性能 CPU 通常将物理主存的一部分划分为特殊的受保护区域，直接充当控制存储器
B. 若采用硬布线（Hardwired）控制器，由于其控制逻辑全部由复杂的物理逻辑门电路直接固化生成，导致其时序控制极其死板，因此在物理层面上绝对无法实现指令流水线（Pipeline）架构
C. 对于采用单周期数据通路设计的 CPU 而言，由于所有的指令都必须在一个时钟周期内严格完成，因此该 CPU 时钟周期的时间跨度，必定是由执行时间最长的那条指令（通常是 Load 指令）的组合逻辑总延迟来决定的
D. 无论采用何种控制器，在执行任何一条无条件转移指令（如 JMP）时，CPU 必定需要经历完整的取指、间址、执行和中断这四个标准的机器周期，以保证程序计数器（PC）的状态一致性

【答案】：C
【解析】：
C 选项完全正确（无歧义）：单周期 CPU 的最大特点是不采用流水线，每条指令独占一个时钟周期。为了保证最复杂的指令（通常是需要依次经历取指、译码、执行、访存、写回组合逻辑的 Load 指令）也能在一个时钟的有效沿到来前跑完所有的门电路延迟，整个全局时钟频率必须被拉低，以匹配这块“最长的短板”
A 选项错误：控制存储器（CM）是微程序控制器的核心部件，它实打实地集成在 CPU 芯片内部的控制器核心中（通常是 ROM），绝对不可能放在芯片外部的慢速主存（RAM）里，否则每次译码都要去主存拿微指令，CPU 会慢得不可思议
B 选项错误：恰恰相反，现代高性能的指令流水线 CPU 几乎全部采用硬布线控制器。因为只有硬连线产生的硬逻辑电信号才能满足 GHz 级别流水线极低的译码延迟要求
D 选项错误：间址周期只有在采用“间接寻址”方式时才会发生；中断周期只有在有外部中断请求时才会发生。一条普通的直接寻址无条件转移指令 `JMP` 只需要取指和执行周期即可

15. 【考点：超标量和动态流水线的基本概念】为了满足深度学习模型推理对算力的极致压榨，现代通用处理器广泛采用了超标量（Superscalar）与动态流水线（乱序执行）技术。关于这些高级流水线技术的实现机制，下列说法错误的是（ ）

A. 为了彻底消除指令流中的所有数据依赖（包括写后读 RAW、写后写 WAW、读后写 WAR）并最大化指令级并行度，现代乱序执行处理器引入了“寄存器重命名”技术，该技术通过增加物理寄存器的数量，从根本上解除了 RAW 冒险导致的计算等待
B. 超标量处理器能够在同一个时钟周期内并发地发射（Issue）多条相互独立的指令，这就要求底层数据通路中必须配置冗余的硬件资源（如多个算术逻辑单元和浮点执行单元）
C. 在采用乱序执行（Out-of-Order Execution）的处理器内部，虽然指令进入计算单元和得出结果的顺序可以与其在原始机器码中的物理顺序不同，但为了保证程序语义和异常处理的正确性，指令的最终结果必须按原顺序提交（In-Order Commit）
D. 超长指令字（VLIW）架构依赖编译器在软件层面静态地挖掘指令级并行性，并将多条互不冲突的操作捆绑成一条极长的指令；而超标量技术则主要依赖 CPU 内部的硬件电路，在程序运行时动态地进行依赖检查和多路发射

【答案】：A
【解析】：
A 选项错误（即正确答案）：“寄存器重命名”技术通过维护映射表和庞大的物理寄存器池，确实能够完美解决由寄存器复用导致的写后写（WAW）和读后写（WAR）假性冒险（名称依赖）。但是，它绝对无法解除读后写（RAW）这一真正的数据依赖。如果 I2 的输入在逻辑上必须依赖 I1 的运算结果，无论你怎么重命名，I2 都必须乖乖等待 I1 的物理电平算出来。因此“从根本上解除了 RAW 冒险”是极度荒谬的描述
B 选项正确：超标量的核心物理基础就是数据通路资源的冗余扩展（例如堆砌多个

ALU/FPU/Load-Store 队列)

C 选项正确：这是乱序执行引擎（如 Tomasulo 算法）的灵魂——重排序缓冲区（Reorder Buffer, ROB）。指令在前端按序发射，在保留站乱序执行，最后必须在 ROB 的尾部严格按原程序顺序提交（In-Order Commit），这是保证精确异常（Precise Exception）和状态机一致性的铁律

D 选项正确：VLIW 是"软件/编译器主导"的并行（如早期的安腾架构），编译器负责把无冲突指令打包；超标量是"硬件主导"的并行，CPU 前端译码器在运行时自己找可以并行的指令

16. **32**．C

对于 load-use 数据冒险，即后一条指令 y 需要使用前一条访存指令 x 从主存中取出的数据，由于指令 y 中所需的数据在指令 x 的访存阶段（MEM）结束时才可使用，即使有转发电路，指令 y 在执行阶段也无法获得所需的数据，只能通过插入 nop 指令或调整指令顺序解决。

17. **32**．B

CPI（每条指令平均时钟周期数）受多种因素影响。发生 Cache 缺失时，CPU 可能需要等待从主存读取数据，这会插入多个空闲周期，从而显著提高程序的平均 CPI。因此 CPI 与 Cache 缺失率密切相关。选项 A、C 和 D 均正确。

18. **33**．A

程序计数器（PC）是专用寄存器，用于存放下一条指令的地址，不属于通用寄存器组，后者由程序员或编译器用于临时数据存储，选项 A 错误。选项 C 具有一定的迷惑性：单周期 CPU 在一个时钟周期内完成整条指令，控制器只需根据操作码一次性生成全部控制信号，逻辑相对简单；而多周期 CPU 需分多个周期执行指令，控制器要在不同阶段动态产生控制信号，设计更复杂——尽管单周期性能较差（受限于最慢指令），但其控制器确实更简单。

19. **07**.【解答】

题干已给出取值和译码阶段每个节拍的功能和有效控制信号，我们应以了解取指阶段中数据通路的信息流动为突破口，读懂每个节拍的功能和有效控制信号，然后应用到解题思路中，包括划分执行步骤、确定完成的功能、需要的控制信号。

先分析题干中提供的示例（本部分解题时不做要求）：

取指令的功能是根据 PC 的内容所指的主存地址，取出指令代码，经过 MDR，最终送至 IR。这部分和后面的指令执行阶段的取操作数、存运算结果的方法是相通的。

C1：(PC)→MAR

在读/写存储器前，必须先将地址（这里为(PC)）送至 MAR。

C2：M(MAR)→MDR, (PC) + 1→PC

读/写的数据必须经过 MDR，指令取出后 PC 自增 1。

C3：(MDR)→IR

然后将读到的 MDR 中的指令代码送至 IR 进行后续操作。

指令"ADD (R1), R0"的操作数一个在主存中，一个在寄存器中，运算结果在主存中。根据指令功能，要读出 R1 的内容所指的主存单元，必须先将 R1 的内容送至 MAR，即(R1)→MAR。而读出的数据必须经过 MDR，即 M(MAR)→MDR。

因此，将 R1 的内容所指的主存单元的数据读出到 MDR 的节拍安排如下：

C5：(R1)→MAR

C6：M(MAR)→MDR

ALU 一端是寄存器 A，MDR 或 R0 中必须有一个先写入 A 中，如 MDR。

C7：(MDR)→A

然后执行加法操作，并将结果送入寄存器 AC。

C8：(A) + (R0)→AC

之后将加法结果写回到 R1 的内容所指的主存单元，注意 MAR 中的内容没有改变。

C9：(AC)→MDR

C10：(MDR)→M(MAR)

有效控制信号的安排并不难，只需看数据是流入还是流出，如流入寄存器 X 就是 Xin，流出寄存器 X 就是 Xout。还需注意其他特殊控制信号，如 PC + 1、Add 等。

于是得到参考答案如下表所示。

时　钟	功　能	有效控制信号
C5	MAR←(R1)	R1out, MARin
C6	MDR←M(MAR)	MemR, MDRinE
C7	A←(MDR)	MDRout, Ain
C8	AC←(A)+(R0)	R0out, ADD, ACin
C9	MDR←(AC)	ACout, MDRin
C10	M(MAR)←(MDR)	MDRoutE, MemW

本题答案不唯一，若在 C6 执行 M(MAR)→MDR 的同时，完成(R0)→A［选择将(R0)写入 A］，并不会发生总线冲突，这种方案可节省 1 个节拍，见下表。

时　钟	功　能	有效控制信号
C5	MAR←(R1)	R1out, MARin
C6	MDR←M(MAR)，A←(R0)	MemR, MDRinE, R0out, Ain
C7	AC←(MDR)+(A)	MDRout, ADD, ACin
C8	MDR←(AC)	ACout, MDRin
C9	M(MAR)←(MDR)	MDRoutE, MemW

20. **08.【解答】**

1）程序员可见寄存器为通用寄存器（R0～R3）和 PC。因为采用了单总线结构，因此若无暂存器 T，则 ALU 的 A、B 端口会同时获得两个相同的数据，使数据通路不能正常工作。

2）ALU 共有 7 种操作，其操作控制信号 ALUop 至少需要 3 位；移位寄存器有 3 种操作，其操作控制信号 SRop 至少需要 2 位。

3）信号 SRout 所控制的部件是一个三态门，用于控制移位器与总线之间数据通路的连接与断开。

4）端口①、②、③、⑤、⑧须连接到控制部件输出端。

5）连线 1，⑥→⑨；连线 2，⑦→④。

6）因为每条指令的长度为 16 位，按字节编址，所以每条指令占用 2 个内存单元，顺序执行时，下条指令地址为(PC)+2。MUX 的一个输入端为 2，可便于执行(PC)+2 操作。

21. **09.【解答】**

1）指令操作码有 7 位，因此最多可定义 $2^7=128$ 条指令。

2）各条指令的机器代码如下：

①“inc R1”的机器码为 0000001 0 01 0 00 0 00，即 0240H。

②“shl R2, R1”的机器码为 0000010 0 10 0 01 0 00，即 0488H。

③“sub R3, (R1), R2”的机器码为 0000011 0 11 1 01 0 10，即 06EAH。

3）各标号处的控制信号或控制信号取值如下：

①0；②mov；③mova；④left；⑤read；⑥sub；⑦mov；⑧SRout。

4）指令“sub R1, R3, (R2)”的执行阶段至少包含 4 个时钟周期；指令“inc R1”的执行阶段至少包含 2 个时钟周期。

22. **10.【解析】**

1）符号标志 SF 表示运算结果的正负性，因此 $SF=F_{15}$。

对于加法运算 A+B→F，若 A、B 为负，且 F 为正，则说明发生溢出；或者，若 A、B 为正，且 F 为负，也说明发生溢出。因此，加运算时，溢出标志 $OF=\overline{A_{15}}\cdot\overline{B_{15}}\cdot F_{15}+A_{15}\cdot B_{15}\cdot\overline{F_{15}}$。

对于减法运算 A－B→F，若 A 为负、B 为正，且 F 为正，则说明发生溢出；或者，若 A 为正、B 为负，且 F 为负，也说明发生溢出。因此，减运算时，溢出标志 $OF=\overline{A_{15}}\cdot B_{15}\cdot F_{15}+$

$A_{15} \cdot \overline{B_{15}} \cdot \overline{F_{15}}$ 。

2）因为在单总线结构中，每一时刻总线上只有一个数据有效，而 ALU 有两个输入端和一个输出端。因此，当 ALU 运算时，需要先用暂存器 Y 缓存其中一个输入端的数据，再通过总线传送另一个输入端的数据。与此同时，ALU 的输出端产生运算结果，但总线正被占用，因此需要暂存器 Z，以缓存 ALU 的输出端数据。

3）由图可知，rs 和 rd 都是 4bit，因此 GPRs 中最多有 $2^4 = 16$ 个通用寄存器；rs 和 rd 来自指令寄存器 IR；rd 表示寄存器编号，应连接地址译码器。

4）取指阶段需要根据程序计数器 PC 取出主存中的指令，并将指令写入指令寄存器 IR 中。控制信号序列如下：

```
①PCout, MARin        //将指令的地址写入 MAR
②Read                //读主存，并将读出的数据写入 MDR
③MDRout, IRin        //将 MDR 的内容写入指令寄存器 IR
```

步骤①需要 1 个时钟周期，步骤②需要 5 个时钟周期，步骤③需要 1 个时钟周期，因此取指令阶段至少需要 7 个时钟周期。

5）图中控制信号由控制部件（CU）产生。指令寄存器 IR 和标志寄存器 FR 的输出信号会连到控制部件的输入端。

23. **03.【解答】**

1）x 的机器码为$[x]_{补} = 1111\ 1101\ 1111\ 1111B$，即指令执行前(R1) = FDFFH，右移 1 位后为 1111 1110 1111 1111B，即指令执行后(R1) = FEFFH。

2）每个时钟周期只能有一条指令进入流水线，从第 5 个时钟周期开始，每个时钟周期都会有一条指令执行完毕，因此至少需要 4 + (5 − 1) = 8 个时钟周期。

3）I_3的 ID 段被阻塞的原因：因为 I_3 与 I_1 和 I_2 都存在数据相关，需等到 I_1 和 I_2 将结果写回寄存器后，I_3 才能读寄存器内容，所以 I_3 的 ID 段被阻塞。I_4 的 IF 段被阻塞的原因：因为 I_4 的前一条指令 I_3 在 ID 段被阻塞，所以 I_4 的 IF 段被阻塞。若 I_4 的 IF 段不被阻塞，则会覆盖指令寄存器的内容，导致 I_3 段的译码结果出错。

注意：要求“按序发射，按序完成”，因此，第二问中下一条指令的 IF 可以和上一条指令的 ID 并行，以免因上一条指令发生冲突而导致下一条指令先执行完。

4）因 2*x 操作有左移和加法两种实现方法，因此 x = x*2 + a 对应的指令序列为

```
I1  LOAD    R1, [x]
I2  LOAD    R2, [a]
I3  SHL     R1          //或者   ADD   R1, R1
I4  ADD     R1, R2
I5  STORE   R2, [x]
```

这 5 条指令在流水线中执行过程如下图所示。

	时间单元																
指 令	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
I1	IF	ID	EX	M	WB												
I2		IF	ID	EX	M	WB											
I3			IF			ID	EX	M	WB								
I4						IF				ID	EX	M	WB				
I5										IF				ID	EX	M	WB

因此执行 x = x*2 + a 语句最少需要 17 个时钟周期。

24. **04.【解答】**

该题为计算机组成原理的综合题型，涉及指令系统、存储管理和 CPU 三部分内容，特别是五段式流水线应引起考生的高度重视。整个指令执行过程中各流水段的时间是相同的，由统一的时钟控制。各流水段在 5.6.1 节中介绍过，这里讨论流水段发生阻塞的情况：

① 若本条指令的源寄存器是上一条指令的目的寄存器，若不采取任何措施，则本条指令取到的将是寄存器的旧值，这就是 RAW 数据冒险。这时有 3 个时钟周期的阻塞，使本指令 ID 应在上一条指令 WB 后。本题中，I1 在 WB 段结束才将结果写回 R4，而 I2 在 ID 段就需要取 R4 的值，如下图所示。

指　令	1	2	3	4	5	6	7	8	9
I_1	IF	ID	EX	M	WB				
I_2		IF				ID	EX	M	WB

第三条指令阻塞了 3 个时钟周期。

② 跳转指令（JMP）（bra）：流水线默认直接取下一条指令，若指令为 JMP 或 JC（条件转移），在没有分支预测的情况下，默认有 3 个时钟周期的阻塞使下一条指令 IF 在分支指令 M 后（分支指令在 M 段才会确定是否更新 PC）。本题中，I6 是分支指令，其后一条指令被阻塞 3 个时钟周期，若跳移条件成立，则转到 I1 执行，反之顺序执行 I7。本题最后一问是分析为什么 I1 的执行不会因为与 I5 的数据相关而阻塞，可以从两个角度来分析：第一，I6 与 I5 存在数据冒险，从而 I6 的 ID 段被阻塞 3 个时钟周期，不论 I6 是不是分支指令，I1 的 IF 段都会被阻塞，从而 ID 段也会相应地推迟。第二，因为 I6 是分支指令，后续指令会阻塞 3 个时钟周期，从而也能消除 I1 和 I5 的数据冒险。

指　令	1	2	3	4	5	6	7	8	9	10	11	12	13
I_5	IF	ID	EX	M	WB								
I_6		IF				ID	EX	M	WB				
I_7/I_7						阻塞	阻塞	阻塞	IF	ID	EX	M	WB

在了解上面的基础知识后，我们再看这道大题。

1）已知计算机 M 采用 32 位定长指令字，即一条指令占 4B，观察表中各指令的地址可知，每条指令的地址差为 4 个地址单位，即 4 个地址单位代表 4B，一个地址单位就代表了 1B，所以该计算机是按字节编址的。

2）在二进制中某数左移两位相当于乘以 4，由该条件可知，数组间的数据间隔为 4 个地址单位，而计算机按字节编址，所以数组 A 中的每个元素占 4B。

3）由表可知，bne 指令的机器代码为 1446FFFAH，根据题目给出的指令格式，后 2B 的内容为 OFFSET 字段，所以该指令的 OFFSET 字段为 FFFAH，用补码表示，值为-6。系统执行到 bne 指令时，PC 自动加 4，PC 的内容为 08048118H，而跳转的目标是 08048100H，两者相差了 18H，即 24 个单位的地址间隔，所以偏移地址的一位即是真实跳转地址的 -24/-6 = 4 位。可知 bne 指令的转移目标地址计算公式为(PC) + 4 + OFFSET×4。

4）由数据相关导致阻塞的指令为第 2、3、4、6 条，因为第 2、3、4、6 条指令都与各自前一条指令发生数据相关。第 6 条指令会发生控制冒险。

当前循环的第 5 条指令与下次循环的第 1 条指令虽然有数据相关，但由于第 6 条指令后有 3 个时钟周期的阻塞，消除了该数据相关（或者解释如下：第 6 条指令因为与第 5 条指令存在数据冒险，导致后续 I1 的执行也相应地推迟，因此消除了该数据冒险）。

25. **05.【解答】**

该题继承了上题中的相关信息，统考中首次引入此种设置，具体考查程序的运行结果、Cache 的大小和命中率的计算，以及磁盘和 TLB 的相关计算，是一道比较综合的题型。2015 年同样出现了 23 分大题的设定，希望读者对其足够重视。

1）R2 中装的是 i 的值，循环条件是 i < N(1000)，即当 i 自增到不满足这个条件时跳出循环，程序结束，所以此时 R2 的值为 1000。

2）Cache 共有 16 行，每块 32 字节，所以 Cache 数据区的容量为 16×32B = 512B。

P 共有 6 条指令，占 24B，小于主存块大小 32B，其起始地址为 0804 8100H，对应一块的开始位置，由此可知所有指令都在一个主存块内。读取第一条指令时会发生 Cache 缺失，因此将 P 所在的主存块调入 Cache 的某一行，以后每次读取指令时，都能在指令 Cache

中命中。因此在 1000 次循环中，只会发生 1 次指令访问缺失，所以指令 Cache 的命中率为(1000×6−1)÷(1000×6) = 99.98%。

3）指令 4 为加法指令，即对应 sum+ = A[i]，当数组 A 中元素的值过大时，会导致这条加法指令发生溢出异常；而指令 2、5 虽然都是加法指令，但它们分别为数组地址的计算指令和存储变量 i 的寄存器进行自增的指令，而 i 最大到达 1000，所以它们都不会产生溢出异常。

只有访存指令可能产生缺页异常，即指令 3 可能产生缺页异常。因为数组 A 在磁盘的一页上，而一开始数组并不在主存中，第一次访问数组时会导致访盘，把 A 调入内存，而以后数组 A 的元素都在内存中，不会导致访盘，所以该程序共访盘一次。

每访问一次内存数据，就查一次 TLB，共访问数组 1000 次，所以此时又访问 1000 次 TLB，还要考虑到第一次访问数组 A，即访问 A[0]时，会多访问一次 TLB（第一次访问 A[0]时会先查一次 TLB，然后产生缺页，处理完缺页中断后，会重新访问 A[0]，此时又查 TLB），所以访问 TLB 的次数一共是 1001 次。

26. 【考点：指令流水线】某计算机采用经典的 5 级流水线（取指 IF、译码/读寄存器 ID、执行/计算有效地址 EX、访存 MEM、写回 WB），支持完全的内部数据转发（Bypass/Forwarding）机制。现有一段连续执行的汇编指令序列如下：

```
I1: ADD R1, R2, R3   ; R1 ← R2 + R3
I2: SUB R4, R1, R5   ; R4 ← R1 - R5
I3: AND R6, R4, R1   ; R6 ← R4 & R1
```

假设 I1 在第 1 个时钟周期进入 IF 段，且不考虑任何 Cache 缺失。当指令 I3 处于 EX（执行）段时，其 ALU 所需的两个源操作数 R4 和 R1 分别是从哪里获取的？（ ）

A. R4 从 EX/MEM 流水段寄存器转发，R1 从 MEM/WB 流水段寄存器转发
B. R4 从 MEM/WB 流水段寄存器转发，R1 从通用寄存器组直接读取
C. R4 从 EX/MEM 流水段寄存器转发，R1 从通用寄存器组直接读取
D. R4 从 MEM/WB 流水段寄存器转发，R1 从 EX/MEM 流水段寄存器转发

【答案】：A
【解析】：

分析指令执行周期：
I1 (ADD)：1(IF) - 2(ID) - 3(EX) - 4(MEM) - 5(WB)
I2 (SUB)：2(IF) - 3(ID) - 4(EX) - 5(MEM) - 6(WB)。注意：I2 依赖 I1 的结果 R1，I1 在第 3 周期算出结果，存入 EX/MEM 寄存器。I2 在第 4 周期的 EX 段正好需要 R1，通过内部数据转发（EX/MEM 到 ALU 输入），没有产生气泡（停顿）
I3 (AND)：3(IF) - 4(ID) - 5(EX) - 6(MEM) - 7(WB)
定位 I3 的 EX 段（第 5 个时钟周期）：
此时，I2 正处于 MEM 段。I2 计算出的 R4 结果目前存放在 `EX/MEM` 流水段寄存器中。因此 I3 所需的 R4 操作数必须由 `EX/MEM` 寄存器前推（转发）给 ALU
此时，I1 正处于 WB 段。I1 计算出的 R1 结果目前存放在 `MEM/WB` 流水段寄存器中，正准备写入寄存器组。因此 I3 所需的 R1 操作数必须由 `MEM/WB` 寄存器前推（转发）给 ALU
综上所述，R4 来自 EX/MEM，R1 来自 MEM/WB，故选 A

27. 【考点：异常和中断机制】关于 CPU 的异常和中断响应机制，下列说法中完全正确的是（ ）

A. 当发生缺页异常（Page Fault）时，硬件在进入异常处理程序前自动保存的程序计数器（PC）内容指向的是引起该异常的指令本身
B. 外部可屏蔽中断（如键盘输入）的检测通常在指令执行周期的任意阶段进行，一旦检测到便立即停止当前指令响应中断
C. 在中断响应周期中，为了保存现场信息，CPU 硬件会自动将所有通用寄存器的内容压入堆栈
D. 为了支持多重中断（中断嵌套），硬件在响应中断并进入中断服务程序时，会自动将中断允许标志（IF）置为 1

【答案】：A
【解析】：

A 正确：缺页异常属于“故障（Fault）”类的内部异常。故障类异常的特点是，处理完成后需要重新执行引起故障的指令。因此，硬件保存的断点（PC）指向的是引起缺页的该条指令本身
B 错误：外部中断属于异步事件，CPU 通常只在每条指令执行周期结束时（而非任意阶段）去检查是否有外部中断请求。内部异常（如除零、缺页）才是在指令执行中途触发响应的
C 错误：在中断响应阶段，硬件自动完成的操作只包括：关中断、保存断点（PC和PSW）、寻找中断向量并更新PC。而保存“所有通用寄存器”内容（即保存现场）属于软件的工作，是由中断服务程序的开头几条指令（Prologue）完成的
D 错误：为了防止在进入中断服务程序的初期（保存现场和中断掩码前）被其他中断破坏现场，硬件在响应中断时会自动关中断（即将 IF 清零）。允许中断嵌套的 IF 置 1 操作是由程序员在保存现场后的软件指令中完成的

28. 【考点：控制器的功能和工作原理】控制器是 CPU 的核心部件，负责产生协调各部件工作的控制信号。对比硬布线控制器与微程序控制器，以下表述正确的是（ ）

A. 微程序控制器中，一条机器指令的执行仅需读取并执行一条对应的微指令即可完成
B. 硬布线控制器的控制信号由组合逻辑电路实时产生，因此其执行速度通常比微程序控制器更快
C. 水平型微指令比垂直型微指令的字长更短，因此存储水平型微程序所需的控制存储器容量更小
D. 微程序控制器在执行指令时，下一条微指令的地址总是完全由当前机器指令的操作码（IR）决定

【答案】：B
【解析】：

A 错误：在微程序控制器中，一条机器指令的执行对应着执行一个微程序，而一个微程序是由多条微指令序列组成的
B 正确：硬布线控制器（纯硬件逻辑）使用门电路等组合逻辑直接翻译指令，没有任何从存储器读取微指令的延迟，所以其速度通常比需要频繁访问控制存储器（CM）的微程序控制器要快
C 错误：水平型微指令的每一位或几个位组合直接对应一个微操作控制信号，可以在一个周期内并行发出多个控制信号，因此它的字长通常比垂直型微指令长得多
D 错误：只有在取指阶段结束后、进入执行阶段的第一条微指令的地址，是由当前机器指令的操作码（IR）映射产生的。随后执行的微指令地址通常是由当前微指令的“下地址字段”给出，或是根据状态标志转移产生的

29. 【考点：多处理器基本概念】现代高级处理器引入了多种技术以提升指令级并行度（ILP）和线程级并行度（TLP）。关于这些架构技术的描述，下列选项中正确的是（ ）

A. 超标量（Superscalar）处理器能在同一个时钟周期内发射多条相互独立的指令，但它必须完全依赖编译器在编译阶段将指令重新排序和打包
B. 细粒度多线程（Fine-grained Multithreading）技术仅在当前线程遇到长延迟事件（如 Cache 缺失）时才切换到另一个线程，以最大化单线程性能
C. 同时多线程（Simultaneous Multithreading, SMT）架构允许在同一个时钟周期内，将来自多个不同线程的指令同时发射到多个不同的功能部件中执行
D. 在典型的多核处理器（Multi-core）架构中，各个核心通常共享 L1 Cache 和 L2 Cache，以实现最高效的缓存一致性同步

【答案】：C
【解析】：

A 错误：超标量处理器通常配备复杂的动态调度硬件（如保留站、重排序缓冲等），能在运行期间由硬件动态地发现并行性并多发射指令。完全依赖编译器静态打包指令的技术被称为超长指令字（VLIW），而非超标量
B 错误：描述的是粗粒度多线程（Coarse-grained）。细粒度多线程是每个时钟周期都在多个线程之间轮换发射指令，它能很好地隐藏短暂停顿，但会牺牲单线程的执行速度
C 正确：同时多线程（SMT，如 Intel 的超线程技术）结合了多线程和超标量技术。它拥有多个线程的上下文，并且在一个时钟周期内，发射逻辑可以从多个线程中挑出就绪的指令，同时派发给下层的多个执行单元（ALU/FPU），极大提高了资源利用率
D 错误：在绝大多数现代多核处理器中，为了保证读取速度，L1 Cache 甚至是 L2 Cache 通常是

每个核心私有的（Private），而 L3 Cache 才是所有核心共享的。缓存一致性协议（如 MESI）主要用于维护私有 Cache 之间的数据一致性

30. 1. 数据冒险与流水线周期分析

数据冒险（RAW）分析：

`I1` 与 `I2` 之间关于寄存器 `R1` 存在数据冒险（`I2` 需读取 `I1` 的写入结果）

`I2` 与 `I3` 之间关于寄存器 `R3` 存在数据冒险（`I3` 需读取 `I2` 的写入结果）

`I3` 与 `I4` 之间关于寄存器 `R5` 存在数据冒险（`I4` 需读取 `I3` 的写入结果）

转发数据源：

`I1` 是一条 Load 指令，其数据只有在 MEM 段结束时才真正从内存中读出就绪。`I2` 在 EX 段需要该数据，因此即使有转发机制，也必须触发一次 Load-Use 阻塞（在 ID 段插入一个气泡，停顿 1 个周期）

停顿 1 个周期后，当 `I2` 进入 EX 段时，`I1` 正好处于 WB 段。因此，`R1` 操作数是从 MEM/WB 流水段寄存器 转发到 EX 段的 ALU 输入端的

总时钟周期：

第 1~5 周期：`I1` 分别处于 IF, ID, EX, MEM, WB

第 2~7 周期：`I2` 在 IF, ID, ID(阻塞), EX, MEM, WB

第 3~8 周期：`I3` 在 IF, IF(被阻塞), ID, EX, MEM, WB（`I3` 的 EX 段可直接接收来自 `I2` 的 EX/MEM 转发，无额外阻塞）

第 4~9 周期：`I4` 在 IF(等待), IF, ID, EX, MEM, WB

完成这 4 条指令共需 9 个时钟周期

2. 数据通路与指令执行过程

EX 段（执行/计算有效地址）：

微操作：将基址寄存器 `R2` 的值与指令中提取的符号扩展后的偏移量（此处为 `0`）进行加法运算，计算出目标主存的有效物理地址

核心部件：主要使用 ALU（算术逻辑单元） 进行加法运算

MEM 段（访存）：

微操作：根据 EX 段计算出的有效地址，向数据存储器发送读控制信号，读取对应内存单元中的数据，并将其存入 MEM/WB 级间流水寄存器中

核心部件：主要使用 Data Memory（数据存储器 / 数据 Cache）

3. 异常和中断机制的检测与响应

异常检测阶段：

算术溢出异常是在执行运算时发生的，因此在流水线的 EX（执行）段 由 ALU 中的硬件逻辑检测到

硬件响应过程： 处理器硬件在响应此类内部异常时，通常自动完成以下三个关键步骤（无需软件干预）：

关中断：将中断允许标志（IF）置为 0，防止在保存现场的过程中被新的更高优先级中断打断

保存断点和程序状态：将导致异常的指令的 PC 值（或下一条指令的 PC）以及当前的程序状态字（PSW）寄存器内容压入系统控制栈或专用的异常寄存器中

识别异常原因并跳转：硬件根据异常类型（算术溢出）生成对应的异常向量地址，将程序计数器（PC）更新为异常处理程序的入口地址，从而转交操作系统接管处理

4. 多处理器基本概念与多线程

SIMD 的加速原理：

SIMD（单指令多数据流）通过使用超宽的向量寄存器和并行的 ALU 执行单元，使得一条机器指令可以同时对多个数据元素（如张量/矩阵的一行或一列）进行相同的操作，实现了数据级并行，极大地减少了取指和译码的开销，从而加速大规模并行计算

硬件多线程的切换策略区别：

粗粒度多线程（Coarse-grained）：仅当当前线程遇到长延迟事件（如 Cache 缺失导致访

存阻塞）时，硬件才会切换执行另一个线程。其单线程执行效率较高，但切换开销相对明显

细粒度多线程（Fine-grained）：在每个时钟周期都会轮换发射不同线程的指令。它能有效隐藏所有长短延迟（包括数据冒险导致的气泡），但由于线程频繁交替，会降低单个线程的绝对执行速度